

## 721. Accounts Merge Medium

Acceptance: 60.8% | [LeetCode](#)

Given a list of `accounts` where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are **emails** representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails **in sorted order**. The accounts themselves can be returned in **any order**.

### Example 1:

```
**Input:** accounts =  
[["John","johnsmith@mail.com","john_newyork@mail.com"],  
 ["John","johnsmith@mail.com","john00@mail.com"],  
 ["Mary","mary@mail.com"],  
 ["John","johnnybravo@mail.com"]]  
**Output:**  
[["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],  
 ["Mary","mary@mail.com"],  
 ["John","johnnybravo@mail.com"]]  
**Explanation:**
```

The first and second John's are the same person as they have the common email "johnsmith@mail.com". The third John and Mary are different people as none of their email addresses are used by other accounts. We could return these lists in any order, for example the answer `[['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']]` would still be accepted.

### Example 2:

```
**Input:** accounts =  
[["Gabe","Gabe0@m.co","Gabe3@m.co","Gabe1@m.co"],  
 ["Kevin","Kevin3@m.co","Kevin5@m.co","Kevin0@m.co"],  
 ["Ethan","Ethan5@m.co","Ethan4@m.co","Ethan0@m.co"],  
 ["Hanzo","Hanzo3@m.co","Hanzo1@m.co","Hanzo0@m.co"],  
 ["Fern","Fern5@m.co","Fern1@m.co","Fern0@m.co"]]  
**Output:**  
[["Ethan","Ethan0@m.co","Ethan4@m.co","Ethan5@m.co"],  
 ["Gabe","Gabe0@m.co","Gabe1@m.co","Gabe3@m.co"],  
 ["Hanzo","Hanzo0@m.co","Hanzo1@m.co","Hanzo3@m.co"],  
 ["Kevin","Kevin0@m.co","Kevin3@m.co","Kevin5@m.co"],  
 ["Fern","Fern0@m.co","Fern1@m.co","Fern5@m.co"]]
```

### Constraints:

- `1 <= accounts.length <= 1000`
- `2 <= accounts[i].length <= 10`
- `1 <= accounts[i][j].length <= 30`
- `accounts[i][0]` consists of English letters.
- `accounts[i][j]` (for `j > 0`) is a valid email.

## Claude Opus Solution

## Accounts Merge — Solution Guide

### 1. Intuition

The key insight is recognizing this as a **graph connectivity problem**:

- Each **email is a node**.
- Two emails are **connected** if they appear in the same account.
- All emails that are transitively connected belong to the **same person** (one connected component).

Once we frame it this way, the problem becomes: **find all connected components**, then attach the owner's name and sort emails within each component.

This is a classic **Union-Find (Disjoint Set Union)** pattern — whenever you need to group items by "belongs together" transitively, DSU is the cleanest tool.

### 2. Approach (Pseudocode)

1. Create a DSU over the ACCOUNTS (indices `0..n-1`).
  - We union account indices, not emails, to keep the DSU small.
2. Build a map: email  $\rightarrow$  first account index that owned it.
  - For each account, for each email:
    - if email already seen  $\rightarrow$  union(current account, previous owner)
    - record email  $\rightarrow$  current account index
3. Group emails by their component root:

- For each (email, account) pair:  
    root = find(account)  
    group[root].append(email)
4. Build the result:
- For each root, take the name (accounts[root][0]) + sorted emails.

### 3. Python Solution

```
from collections import defaultdict
from typing import List

class DSU:
    """Disjoint Set Union with path compression + union by rank."""

    def __init__(self, n: int):
        # Each node starts as its own parent (its own set)
        self.parent = list(range(n))
        self.rank = [0] * n # approximate tree height for balancing

    def find(self, x: int) -> int:
        # Path compression: flatten the tree as we search
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a: int, b: int) -> None:
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return # already in same set
        # Union by rank: attach smaller tree under larger one
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1

class Solution:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        n = len(accounts)
        dsu = DSU(n)

        # Step 1: Map each email to the FIRST account index that claimed it.
        # When we see an email again, we know those two accounts belong
        # to the same person -> union their indices.
        email_to_acct = {}
        for i, account in enumerate(accounts):
            for email in account[1:]: # skip the name at index 0
                if email in email_to_acct:
                    # This email links account `i` with a previous account
                    dsu.union(i, email_to_acct[email])
                else:
                    email_to_acct[email] = i

        # Step 2: Group all emails under their component's root account.
        # Using a set avoids duplicate emails within the same component.
        groups = defaultdict(set)
        for email, acct in email_to_acct.items():
            root = dsu.find(acct)
            groups[root].add(email)

        # Step 3: Build result -> [name, sorted emails...]
        result = []
        for root, emails in groups.items():
            name = accounts[root][0] # name lives at the root account
            result.append([name] + sorted(emails))

        return result
```

## 4. Complexity Analysis

Let **N** = number of accounts and **M** = max emails per account. Total emails  $\approx \mathbf{N \cdot M}$ .

**Time:  $O(N \cdot M \cdot \log(N \cdot M))$**  - Building the DSU and processing all emails: each `union/find` is nearly  $O(1)$  amortized —  $\alpha(N)$  (inverse Ackermann), effectively constant. - Total union-find work:  **$O(N \cdot M \cdot \alpha(N))$** . - The dominant cost is **sorting** the emails. In the worst case (all emails in one component) we sort  $N \cdot M$  emails  $\rightarrow O(N \cdot M \cdot \log(N \cdot M))$ .

**Space:  $O(N \cdot M)$**  - `email_to_acct` map and `groups` store all emails. - DSU arrays are  $O(N)$ .

## 5. Alternative Approaches

### A. DFS / BFS on an Email Graph (Reference Solution 1)

- Build an adjacency list where emails in the same account are connected (often in a "star" pattern with the first email).
- Run DFS/BFS from each unvisited email to collect a connected component.
- **Same time/space complexity**, but uses an explicit graph traversal. Watch out for **recursion depth limits** in Python with large components — BFS is safer.

### B. Union-Find on Emails Directly

- Instead of unioning *account indices*, you can union *emails* themselves (map each email to an integer id).
- Slightly more bookkeeping (need an email $\rightarrow$ id map), but conceptually clean.
- Same complexity.

#### Why the Account-Index DSU is preferred here:

- The DSU size is just **N ( $\leq 1000$ )** instead of  $N \cdot M$ , keeping it small.
- The name is trivially recovered via `accounts[root][0]`, since all accounts in a component share the same name.

### Quick Comparison

| Approach                      | DSU size       | Recursion risk        | Clarity |
|-------------------------------|----------------|-----------------------|---------|
| Union-Find on account indices | $O(N)$         | None                  | ☆☆☆     |
| DFS on email graph            | —              | Yes (deep components) | ☆☆      |
| Union-Find on emails          | $O(N \cdot M)$ | None                  | ☆☆      |

**Recommendation:** Use Union-Find on account indices — it's the most space-efficient and avoids recursion pitfalls.

## Codex Solution

### 1. Intuition

This is a **connected components** problem.

Think of accounts/emails as being connected:

- If two accounts share an email, they belong to the same person.
- If account A shares an email with account B, and account B shares another email with account C, then all three accounts belong to the same person.

So we need to group accounts that are connected by shared emails.

A clean way to do this is with **Disjoint Set Union**, also called **Union-Find**.

Key idea:

- Treat each account as a node.
- If two accounts share an email, union them into the same group.
- After processing all accounts, collect emails by each group's root account.

## 2. Approach

### Pseudocode

```
Create DSU with one node per account

email_to_account = empty hashmap

For each account index i:
    For each email in accounts[i]:
        If email has not been seen before:
            email_to_account[email] = i
        Else:
            previous_account = email_to_account[email]
            union(i, previous_account)

Create merged_emails hashmap:
    root_account_index -> list of emails

For each email, account_index in email_to_account:
    root = find(account_index)
    Add email to merged_emails[root]

answer = []

For each root in merged_emails:
    Sort the emails
    name = accounts[root][0]
    Add [name] + sorted_emails to answer

Return answer
```

## 3. Python Solution

```
from typing import List
from collections import defaultdict

class DSU:
    """
    Disjoint Set Union / Union-Find data structure.

    Supports:
    - find(x): returns the representative/root of x's group
    - union(a, b): merges the groups containing a and b
    """

    def __init__(self, n: int):
        # Initially, every account is its own parent.
        self.parent = list(range(n))

        # rank[x] is an upper bound on the tree height.
        # Used to keep the DSU tree shallow.
        self.rank = [1] * n

    def find(self, x: int) -> int:
        """
        Finds the root representative of x.

        Uses path compression:
        while finding the root, make every node on the path
        point directly to the root.
        """
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])

        return self.parent[x]

    def union(self, a: int, b: int) -> None:
        """
```

```

Merges the components containing a and b.
"""
root_a = self.find(a)
root_b = self.find(b)

# Already in the same component.
if root_a == root_b:
    return

# Union by rank:
# attach the smaller tree under the larger tree.
if self.rank[root_a] > self.rank[root_b]:
    self.parent[root_b] = root_a
elif self.rank[root_a] < self.rank[root_b]:
    self.parent[root_a] = root_b
else:
    self.parent[root_b] = root_a
    self.rank[root_a] += 1

```

**class Solution:**

```

def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
    n = len(accounts)

    # DSU over account indices.
    dsu = DSU(n)

    # Maps each email to the first account index where we saw it.
    email_to_account = {}

    # Step 1: Union accounts that share an email.
    for account_index, account in enumerate(accounts):
        # account[0] is the name, emails start from index 1.
        for email in account[1:]:
            if email not in email_to_account:
                # First time seeing this email.
                email_to_account[email] = account_index
            else:
                # This email was seen before, so both accounts
                # must belong to the same person.
                previous_account_index = email_to_account[email]
                dsu.union(account_index, previous_account_index)

    # Step 2: Group emails by the root account of their component.
    root_to_emails = defaultdict(list)

    for email, account_index in email_to_account.items():
        root = dsu.find(account_index)
        root_to_emails[root].append(email)

    # Step 3: Build the final answer.
    merged_accounts = []

    for root, emails in root_to_emails.items():
        # All accounts in the same component have the same name,
        # so we can use the root account's name.
        name = accounts[root][0]

        # Emails must be returned in sorted order.
        merged_accounts.append([name] + sorted(emails))

    return merged_accounts

```

## 4. Complexity Analysis

Let:

- $n$  be the number of accounts
- $m$  be the maximum number of emails per account

- $E$  be the total number of email occurrences across all accounts  
 $E \leq n * m$
- $U$  be the number of unique emails

### Time Complexity

Union operations:

$$O(E * \alpha(n))$$

where  $\alpha(n)$  is the inverse Ackermann function, which is effectively constant in practice.

Grouping emails:

$$O(U * \alpha(n))$$

Sorting emails inside each merged account:

$$O(U \log U)$$

in the worst case, when all emails belong to one merged account.

Therefore, total time complexity is:

$$O(E * \alpha(n) + U \log U)$$

Since  $E \leq n * m$  and  $U \leq E$ , this is commonly written as:

$$O(nm \log(nm))$$

because sorting dominates.

### Space Complexity

We store:

- DSU parent and rank arrays:  $O(n)$
- email-to-account map:  $O(U)$
- grouped emails:  $O(U)$

Total space complexity:

$$O(n + U)$$

or, in terms of  $n$  and  $m$ :

$$O(nm)$$

## 5. Alternative Approaches

### 1. DFS / BFS Graph Traversal

Build a graph where:

- Each email is a node.
- Emails in the same account are connected.
- For example, connect every email in an account to the first email.

Then run DFS/BFS to find connected components of emails.

This approach is also very common and has similar complexity:

$$\begin{aligned} \text{Time: } & O(nm \log(nm)) \\ \text{Space: } & O(nm) \end{aligned}$$

### 2. DSU on Emails Instead of Accounts

Instead of unioning account indices, we can assign each unique email an ID and union emails directly.

This also works well, but requires mapping every email to an integer ID. The account-index DSU version is slightly simpler because the number of accounts is already known.